

Antony Polukhin <antoshkka@gmail.com>, <antoshkka@yandex-team.ru>

Date: 2016-11-10

Constexpr for std::complex

Significant changes to [P0415R0](#) are marked with blue.

I. Introduction and Motivation

There is a request in the "C++ Standard Library Active Issues List" for a [constexpr for various std::complex arithmetic and value operators](#). Currently the following code fails to compile:

```
#include <array>
#include <algorithm>

int main() {
    // OK
    constexpr std::complex<double> c1 { 1.0, 0.0 };
    constexpr std::complex<double> c2 {};

    // Failure: arithmetic operations on complex are not constexpr
    constexpr auto c3 = -c1 + c2 / 100.0;
}
```

This proposal attempts to solve the issue.

A proof of concept implementation for is available at: [constexpr complex](#).

II. Impact on the Standard

This proposal is a pure library extension. It proposes changes to the existing header `<complex>` such that the changes do not break existing code and do not degrade performance. It does not require any changes in the core language and it can be implemented in standard C++.

III. Analysis of existing `<complex>` implementations and making them constexpr.

libstdc++ uses `__complex__` extension. That extension is already being used in constant expressions.

libc++ implements all the algorithms for `std::complex` from scratch. libc++ uses few functions from `<cmath>` (mostly `isnan`, `isinf`, `logb` and `scalbn`) for operations `+`, `-`, `*`, `/`, `norm`, `conj`.

Operations other than `+`, `-`, `*`, `/`, `norm`, `conj` use a lot of functions from `<cmath>`. It's unreasonable to mark them with `constexpr` right now.

`abs(const complex<T>&)` seems highly usable however it requires either `sqrt` or `hypot` to be `constexpr`. Forcing complicated math functions to be `constexpr` intrinsics may be a burden for compiler developers.

Adding `constexpr` to libc++ according to the wording in section IV of this paper resulted in [a `<complex>` header](#) that works well on GCC-6. `__builtin_logb`, `__builtin_isnan` and `__builtin_scalbln` are recognised by Clang, but they are not usable in constant expressions right now. [Header](#) with a hand-made naïve `logb`, `isnan` and `scalbn` became usable on CLANG-3.6 too.

IV. Proposed wording relative to the Working Draft N4594.

A. Modifications to "Header `<complex>` synopsis" [`complex.syn`]

```
namespace std {
    template<class T> class complex;
    template<> class complex<float>;
    template<> class complex<double>;
    template<> class complex<long double>;
```

```

// 26.5.6, operators:
template<class T> constexpr complex<T> operator+(const complex<T>&, const complex<T>&);
template<class T> constexpr complex<T> operator+(const complex<T>&, const T&);
template<class T> constexpr complex<T> operator+(const T&, const complex<T>&);
template<class T> constexpr complex<T> operator-(const complex<T>&, const complex<T>&);
template<class T> constexpr complex<T> operator-(const complex<T>&, const T&);
template<class T> constexpr complex<T> operator-(const T&, const complex<T>&);
template<class T> constexpr complex<T> operator*(const complex<T>&, const complex<T>&);
template<class T> constexpr complex<T> operator*(const complex<T>&, const T&);
template<class T> constexpr complex<T> operator*(const T&, const complex<T>&);
template<class T> constexpr complex<T> operator/(const complex<T>&, const complex<T>&);
template<class T> constexpr complex<T> operator/(const complex<T>&, const T&);
template<class T> constexpr complex<T> operator/(const T&, const complex<T>&);
template<class T> constexpr complex<T> operator+(const complex<T>&);
template<class T> constexpr complex<T> operator-(const complex<T>&);

template<class T> constexpr bool operator==(const complex<T>&, const complex<T>&);
template<class T> constexpr bool operator==(const complex<T>&, const T&);
template<class T> constexpr bool operator==(const T&, const complex<T>&);
template<class T> constexpr bool operator!=(const complex<T>&, const complex<T>&);
template<class T> constexpr bool operator!=(const complex<T>&, const T&);
template<class T> constexpr bool operator!=(const T&, const complex<T>&);

template<class T, class charT, class traits>
basic_istream<charT, traits>& operator>>(basic_istream<charT, traits>&, complex<T>&);

template<class T, class charT, class traits>
basic_ostream<charT, traits>& operator<<(basic_ostream<charT, traits>&, const complex<T>&);

// 26.5.7, values:
template<class T> constexpr T real(const complex<T>&);
template<class T> constexpr T imag(const complex<T>&);
template<class T> T abs(const complex<T>&);
template<class T> T arg(const complex<T>&);
template<class T> constexpr T norm(const complex<T>&);

template<class T> constexpr complex<T> conj(const complex<T>&);
template<class T> complex<T> proj(const complex<T>&);
template<class T> complex<T> polar(const T&, const T& = 0);

// 26.5.8, transcendentals:
template<class T> complex<T> acos(const complex<T>&);
template<class T> complex<T> asin(const complex<T>&);
template<class T> complex<T> atan(const complex<T>&);
template<class T> complex<T> acosh(const complex<T>&);
template<class T> complex<T> asinh(const complex<T>&);
template<class T> complex<T> atanh(const complex<T>&);
template<class T> complex<T> cos (const complex<T>&);
template<class T> complex<T> cosh (const complex<T>&);
template<class T> complex<T> exp (const complex<T>&);
template<class T> complex<T> log (const complex<T>&);
template<class T> complex<T> log10(const complex<T>&);

template<class T> complex<T> pow (const complex<T>&, const complex<T>&);
template<class T> complex<T> pow (const T&, const complex<T>&);
template<class T> complex<T> pow (const complex<T>&, const T&);

template<class T> complex<T> sin(const complex<T>&);
template<class T> complex<T> sinh(const complex<T>&);
template<class T> complex<T> sqrt(const complex<T>&);
template<class T> complex<T> tan(const complex<T>&);
template<class T> complex<T> tanh(const complex<T>&);

// 26.5.10, complex literals:

inline namespace literals {
inline namespace complex_literals {
constexpr complex<long double> operator""il(long double);
constexpr complex<long double> operator""il(unsigned long long);
constexpr complex<double> operator""i(long double);
constexpr complex<double> operator""i(unsigned long long);
constexpr complex<float> operator""if(long double);
constexpr complex<float> operator""if(unsigned long long);
}
}

```

```
}
```

B. Modifications to "Class template complex" [complex]

```
namespace std {
    template<class T>
    class complex {
    public:
        typedef T value_type;
        constexpr complex(const T& re = T(), const T& im = T());
        constexpr complex(const complex&);
        template<class X> constexpr complex(const complex<X>&);
        constexpr T real() const;
        constexpr void real(T);

        constexpr T imag() const;
        constexpr void imag(T);

        constexpr complex<T>& operator= (const T&);
        constexpr complex<T>& operator+=(const T&);
        constexpr complex<T>& operator-=(const T&);
        constexpr complex<T>& operator*=(const T&);
        constexpr complex<T>& operator/=(const T&);

        constexpr complex& operator=(const complex&);
        template<class X> constexpr complex<T>& operator= (const complex<X>&);
        template<class X> constexpr complex<T>& operator+=(const complex<X>&);
        template<class X> constexpr complex<T>& operator-=(const complex<X>&);
        template<class X> constexpr complex<T>& operator*=(const complex<X>&);
        template<class X> constexpr complex<T>& operator/=(const complex<X>&);
    };
}
```

The class complex describes an object that can store the Cartesian components, real() and imag(), of a complex number.

C. Modifications to "complex specializations" [complex.special]

```
namespace std {
    template<> class complex<float> {
    public:
        typedef float value_type;
        constexpr complex(float re = 0.0f, float im = 0.0f);
        constexpr explicit complex(const complex<double>&);
        constexpr explicit complex(const complex<long double>&);

        constexpr float real() const;
        constexpr void real(float);
        constexpr float imag() const;
        constexpr void imag(float);

        constexpr complex<float>& operator= (float);
        constexpr complex<float>& operator+=(float);
        constexpr complex<float>& operator-=(float);
        constexpr complex<float>& operator*=(float);
        constexpr complex<float>& operator/=(float);

        constexpr complex<float>& operator=(const complex<float>&);
        template<class X> constexpr complex<float>& operator= (const complex<X>&);
        template<class X> constexpr complex<float>& operator+=(const complex<X>&);
        template<class X> constexpr complex<float>& operator-=(const complex<X>&);
        template<class X> constexpr complex<float>& operator*=(const complex<X>&);
        template<class X> constexpr complex<float>& operator/=(const complex<X>&);
    };

    template<> class complex<double> {
    public:
        typedef double value_type;
        constexpr complex(double re = 0.0, double im = 0.0);
        constexpr complex(const complex<float>&);
        constexpr explicit complex(const complex<long double>&);

        constexpr double real() const;
        constexpr void real(double);
        constexpr double imag() const;
        constexpr void imag(double);
    };
}
```

```

constexpr complex<double>& operator= (double);
constexpr complex<double>& operator+=(double);
constexpr complex<double>& operator-=(double);
constexpr complex<double>& operator*=(double);
constexpr complex<double>& operator/=(double);

constexpr complex<double>& operator=(const complex<double>&);
template<class X> constexpr complex<double>& operator= (const complex<X>&);
template<class X> constexpr complex<double>& operator+=(const complex<X>&);
template<class X> constexpr complex<double>& operator-=(const complex<X>&);
template<class X> constexpr complex<double>& operator*=(const complex<X>&);
template<class X> constexpr complex<double>& operator/=(const complex<X>&);
};

template<> class complex<long double> {
public:
    typedef long double value_type;
    constexpr complex(long double re = 0.0, long double im = 0.0);
    constexpr complex(const complex<float>&);
    constexpr explicit complex(const complex<double>&);

    constexpr long double real() const;
    constexpr void real(long double);
    constexpr long double imag() const;
    constexpr void imag(long double);

    constexpr complex<long double>& operator= (long double);
    constexpr complex<long double>& operator+=(long double);
    constexpr complex<long double>& operator-=(long double);
    constexpr complex<long double>& operator*=(long double);
    constexpr complex<long double>& operator/=(long double);

    constexpr complex<long double>& operator=(const complex<long double>&);
    template<class X> constexpr complex<long double>& operator= (const complex<X>&);
    template<class X> constexpr complex<long double>& operator+=(const complex<X>&);
    template<class X> constexpr complex<long double>& operator-=(const complex<X>&);
    template<class X> constexpr complex<long double>& operator*=(const complex<X>&);
    template<class X> constexpr complex<long double>& operator/=(const complex<X>&);
};
}

```

D. Modifications to "Additional overloads" [cmplx.over]

The following function templates shall have additional overloads:

arg	norm
conj	proj
imag	real

where norm, conj, imag and real are constexpr overloads.

The additional overloads shall be sufficient to ensure:

[Note for the editor: Following lines in [cmplx.over] do not change.- end note]

E. Modifications to remaining parts of "Complex numbers" [complex.*]

Note for the editor: All the functions marked with *constexpr* in previous paragraphs of this document must be accordingly marked with *constexpr* in detailed descriptions. For brevity only modifications to "complex member functions" [complex.members] are shown in this paper.

```

template<class T> constexpr complex(const T& re = T(), const T& im = T());
Effects: Constructs an object of class complex.
Postcondition: real() == re && imag() == im.

```

```

constexpr T real() const;
Returns: The value of the real component.

```

```

constexpr void real(T val);
Effects: Assigns val to the real component.

```

```

constexpr T imag() const;
Returns: The value of the imaginary component.

```

`constexpr` void imag(T val);
Effects: Assigns val to the imaginary component.

F. Feature-testing macro

For the purposes of SG10, we recommend the feature-testing macro name `__cpp_lib_constexpr_complex`.

V. Revision History

Revision 1:

- Added [cmplx.over] changes.

Revision 0:

- Initial proposal.

VI. References

[N4594] Working Draft, Standard for Programming Language C++. Available online at www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/n4594.pdf.

[Antony Polukhin] Proof of concept implementation. Available online at https://github.com/apolukhin/apolukhin.github.io/tree/master/papers/constexpr_complex/.

[LWG2693] `constexpr` for various `std::complex` arithmetic and value operators. Available online at <http://www.open-std.org/jtc1/sc22/wg21/docs/lwg-active.html#2693>.

VII. Acknowledgements

Walter E. Brown pointed me at the LWG 2693 issue, provided numerous comments, corrections, and suggestions for this proposal.